

Development Roadmap Document SentinelCut — AI-Powered Audio/Video Profanity Censorship Platform

Document Version: 1.0 Status: Final for Implementation Based On: PRD v1.0 (§9 MoSCoW, §10 Release Planning), System Architecture v1.0, Database Design v1.0, Folder Structure v1.1, API Design v1.0, AI Pipeline v1.0, UI/UX Design v1.0

1. Purpose

This document sequences everything specified in Documents 2-10 into a concrete, sprint-by-sprint build order for an academic-timeline team. It answers the question none of the prior documents answer on their own: *in what order do we actually build this, and what is true and demoable at the end of each sprint*. Every sprint below ends in a working, demoable increment — not a partial slice that only makes sense once three more sprints land — because an academic project is graded/demoed at fixed checkpoints (PRD §10: mid-semester demo, pre-final demo, final submission), not at an arbitrary release date.

2. Planning Assumptions

Assumption	Value	Rationale
Project duration	16 weeks (one academic semester)	Standard college mini-project window
Sprint length	2 weeks	Gives 8 sprints total — enough granularity for weekly progress reporting (per Master Prompt's "Weekly Progress Report" deliverable) without excessive ceremony for a small team
Team size	3-5 students	Typical mini-project team size; role split assumed in Section 8, but the sprint plan degrades gracefully to a smaller team by extending sprint length rather than cutting scope
Infra decisions	Already finalized (Tech Stack Justification v1.0)	No time is spent in this roadmap re-evaluating ASR/DB/hosting choices — that debate is closed
Definition of "demoable"	The feature works end-to-end through the real UI against the real backend, not a mocked/hardcoded state	A sprint is not "done" if only an isolated unit test passes; per PRD §10 the checkpoints are live demos

3. Build-Order Dependency Rationale

Before assigning work to sprints, the dependency chain across Documents 5-10 needs to be made explicit, since several modules *look* independent but are not:

```

Database schema (Doc 6)
└─> Repository layer (Doc 7: backend/app/repositories/)
    └─> Service layer (Doc 7: backend/app/services/)
        └─> API routes (Doc 8)
            └─> Frontend screens (Doc 10) consume API contract
└─> AI Pipeline modules (Doc 9: ai-pipeline/) – buildable in parallel,
    since Doc 9's interfaces (SpeechRecognitionService,
    ProfanityDetectionService) do not depend on the API layer at all,
    only on the DB schema for what a "detection instance" must contain
    └─> Celery task wiring (backend/app/workers/) connects
        the AI Pipeline to the Job/Detection tables
        └─> Timeline Editor (Doc 10 §9) is the
last frontend screen buildable, since it
needs real detection data, not just an
upload ack

```

Consequence for sequencing: the AI Pipeline (Document 9) and the backend CRUD/auth layer can and should be built **in parallel by different team members from Sprint 1 onward**, both against the fixed Database Design schema (Document 6) as the shared contract — this is the single highest-leverage scheduling decision in this roadmap, and is why Sprint 1-2 below split the team across those two tracks rather than building strictly top-to-bottom.

4. Milestone Alignment (PRD §10 → Sprints)

PRD Release	PRD Target Milestone	Sprints	This Document's Milestone Name
R1 — MVP	Mid-semester demo	Sprints 1-4 (Weeks 1-8)	Milestone A: Core Loop Works
R2 — Usability & Control	Pre-final demo	Sprints 5-6 (Weeks 9-12)	Milestone B: User Control
R3 — Polish & Admin	Final submission/demo	Sprints 7-8 (Weeks 13-16)	Milestone C: Production-Shaped Polish
R4+ — Future Scope	Post-graduation	Not scheduled	Documented separately in Document 15: Future Scope

5. Sprint 0 — Setup (Pre-Sprint, ~3-4 days before Sprint 1)

Not a numbered sprint against the 16-week budget, but must complete before Sprint 1 starts.

- Initialize monorepo per Folder Structure v1.1 exactly (`frontend/`, `backend/`, `ai-pipeline/`, `database/`, `infra/`, `docs/`, `scripts/`, `tests/`).
- Provision Supabase project (Postgres + Auth + Storage), Upstash Redis instance, Vercel + Render/Railway accounts (Tech Stack §11).
- Run initial Alembic migration generating all tables from Database Design v1.0 §4 verbatim.
- Seed `profanity_dictionary` with an initial curated English + Hindi + Hinglish slang list (AI Pipeline §8 data source) via `scripts/seed_database.sh`.
- Stand up CI skeleton (`.github/workflows/*-ci.yml`) running lint + placeholder test on every PR from day one — cheap to add now, expensive to retrofit in Sprint 6.
- Copy all 10 finalized documents into `docs/` per the naming convention in Folder Structure §9.

Definition of Done: `docker-compose up` brings up Postgres+Redis+backend+frontend locally with zero errors; CI is green on an empty commit.

6. Milestone A — Core Loop Works (Sprints 1-4)

Sprint 1 (Weeks 1-2): Auth + Upload Skeleton + ASR Proof-of-Concept

Two parallel tracks, per Section 3.

Backend/Frontend track:

- Implement `AuthService`/`user_repository.py`/`auth_routes.py` for email/password signup+login only (Google OAuth and guest mode deferred to Sprint 2 — they're additive, not blocking).
- Implement `FileService`/`file_repository.py`/`file_routes.py` for `POST /files/upload` and `GET /files/{id}` only, writing to Supabase Storage.
- Frontend: `(auth)` route group screens (Doc 10 §6, email/password paths only), `(app)` shell layout (Navbar/Sidebar), bare Upload page (dropzone + validation only, no configuration panel yet).

AI Pipeline track:

- Implement Stage 1 (`audio_extractor.py`) and Stage 2 (`faster_whisper_engine.py`) behind `SpeechRecognitionService`, tested standalone via a CLI script against fixture files — **no Celery/API integration yet**. Goal is proving Faster-Whisper's accuracy and latency on the team's actual target hardware before anything else is built on top of it, since this is the single highest-risk technical assumption in the whole project (per Tech Stack §3).

Definition of Done: A user can sign up, log in, and upload a file that lands in Storage with a `(files)` row; separately, a command-line script transcribes a sample Hindi/Hinglish clip with word-level timestamps and prints recognizable output.

Sprint 2 (Weeks 3-4): Job Queue + Full ASR/Detection Pipeline (Automatic Mode Only)

- Wire Celery/RQ (`(backend/app/workers/celery_app.py)`, `(tasks/processing_task.py)`) so `(POST /files/{id}/process)` actually enqueues and a worker dequeues.
- Complete AI Pipeline Stages 3-7 (language detection, normalization, dictionary matching, context scoring, confidence aggregation) for **Automatic detection mode only** — Custom/Combined modes deferred to Milestone B per the PRD's own MoSCoW (§9: Custom/Combined is a "Should Have," R2).
- Persist `(detections)` rows; implement `(GET /files/{id}/detections)` and `(GET /files/{id}/transcript)`.
- Add Guest Mode (`(POST /auth/guest-session)`) and Google OAuth (`(POST /auth/google)`) to the auth track — both are Must-Haves for R1 per PRD §9 but were reasonably deferred from Sprint 1 since they're additive to an already-working email/password path.
- Frontend: WebSocket/polling status hook (`(useJobStatus.js)`), Processing Status page (Doc 10 §8.3).

Definition of Done: Uploading a file through the real UI results in a real, worker-produced set of detection instances visible via `(GET /files/{id}/detections)` (Postman/curl-level verification is acceptable here; the Timeline Editor UI to view them lands next sprint).

Sprint 3 (Weeks 5-6): Timeline Editor (Read + Edit, No Regeneration Yet) + Beep/Mute Rendering

- Frontend: build the Timeline Editor screen (Doc 10 §9.1-9.4) against the now-real `(/detections)` and `(/transcript)` endpoints — transcript rendering, confidence-color highlighting, detail panel, dismiss/delete, manual-add. This is correctly sequenced as the last major frontend screen in Milestone A because it has the longest dependency chain (Section 3).
- Backend: `(PATCH /detections/{id})`, `(PATCH /files/{id}/detections/batch)`, `(POST /files/{id}/detections)`.
- AI Pipeline: implement Stage 8 (`(audio_editor.py)`) for Beep and Mute only (Custom Sound deferred — it's an R2 "Should Have" per PRD §9) and Stage 9 (`(media_muxer.py)`).
- Wire `(POST /files/{id}/regenerate)` end-to-end, calling only Stages 8-9 (AI Pipeline §12/§14) — this is the first point in the project where the "regeneration is

cheap" architectural decision actually gets exercised and proven, not just documented.

Definition of Done: A user can review AI-detected profanity with visible confidence colors, dismiss a false positive, add a missed one, click Regenerate, and download a video with an audible beep at the correct timestamp.

Sprint 4 (Weeks 7-8): Export Formats + Dashboard + Mid-Semester Demo Hardening

- Backend: `GET /files/{id}/exports`, signed-URL download endpoint, generating `.srt`/`.txt`/`.json` exports alongside the censored media (SRS-FR-EXPT-03/04/05).
- Frontend: Dashboard (Doc 10 §7) wired to `GET /dashboard/summary`; inline preview player in the Timeline Editor (Doc 10 §9.7).
- Bug-fix/hardening pass: run the integration test from AI Pipeline §18 against 5-10 real sample files spanning English/Hindi/Hinglish; fix any sync-drift or normalization gaps found.
- Rehearse the demo script against PRD §6.1's Primary Journey exactly (upload → automatic detect → review → regenerate → download).

Definition of Done — Milestone A / Mid-Semester Demo: Every "Must Have" row in PRD §9's MoSCoW table for R1 works live, end-to-end, through the deployed (not just local) environment.

7. Milestone B — User Control (Sprints 5-6)

Sprint 5 (Weeks 9-10): Custom/Combined Detection + Presets + Custom Sound Replacement

- AI Pipeline: extend the detection-config merge logic (AI Pipeline §14, `merge_with_custom_words`) to actually support Custom and Combined modes, reusing the Stage 4-5 normalization/matching code path as designed.
- Backend: `presets` CRUD routes (API Design §9); Custom Sound support in `audio_editor.py` (AI Pipeline §12, step 4's loop/trim logic).
- Frontend: Upload page's full Configuration Panel (Doc 10 §8.2, mode selector + preset dropdown + custom-sound sub-upload).

Definition of Done: A user can define a custom word list, save it as a named preset, reuse it on a later upload, and choose a custom audio clip as the replacement sound.

Sprint 6 (Weeks 11-12): Subtitles, History/Search, Confidence Filtering + Pre-Final Demo Hardening

- Backend: `.srt` export generation with censorship markers (SRS-FR-EXPT-03); `GET /files` search/filter/sort query params (API Design §4.3).

- Frontend: History page (Doc 10 §10); Timeline Editor confidence-threshold slider (Doc 10 §9.5); undo/redo stack (Doc 10 §9.6, at least 10 actions per SRS-FR-TLE-09).
- Begin the accuracy benchmark pass (AI Pipeline §18, `scripts/benchmark_asr.py` / `benchmark_detection.py`) against a labeled test set, producing the actual recall/precision numbers referenced in PRD §4.6 AC-1 — this needs a full sprint of lead time before the final demo, not a last-week scramble, since tuning `CONTEXT_SIMILARITY_THRESHOLD` (AI Pipeline §19) based on these results may require re-running the benchmark multiple times.

Definition of Done — Milestone B / Pre-Final Demo: Every "Should Have" row in PRD §9's MoSCoW for R2 works live; a written accuracy report exists with real recall/precision numbers against the internal test set.

8. Milestone C — Production-Shaped Polish (Sprints 7-8)

Sprint 7 (Weeks 13-14): Admin Panel + Rate Limiting/Virus Scan + Dark Mode

- Backend: all `/admin/*` routes (API Design §10) — user management, dictionary CRUD, files overview, feedback; `admin_logs` audit writes on every mutation (SRS-FR-ADM-05).
- Security hardening pass: rate limiting on `/auth/*` and `/files/upload` (SRS-NFR-SEC-06), malware scanning integration on upload (SRS-FR-UPLD-07) — both explicitly scheduled here rather than earlier, since they are R3 "Could Have" items per PRD §9 and gate nothing else in Milestones A/B.
- Frontend: all five Admin sub-screens (Doc 10 §12); global dark-mode token toggle (Doc 10 §3.5).

Definition of Done: An admin user can disable a user account, edit the profanity dictionary and see the change reflected in new uploads within the documented cache TTL (AI Pipeline §8), and view basic analytics — all while the rest of the app renders correctly in both themes.

Sprint 8 (Weeks 15-16): Credits/Storage UI, Full Regression, Documentation Finalization, Final Demo

- Frontend: Storage-usage progress bar and Credits display polish (already data-available from Sprint 4's Dashboard, this sprint adds the `credits_ledger`-backed consumption logic if not already wired); Settings/Profile screens (Doc 10 §11), including the account-deletion "Danger Zone" flow (SRS-NFR-LEGAL-02).
- Full regression pass against every AC in the PRD (§4.1-§4.10) and every SRS-NFR category (§5.1-§5.7) — this is a dedicated QA sprint, not a bug-fix-as-you-go sprint, and should follow the Test Plan/Test Cases/QA Checklist deliverables (Document 13).

- Finalize all remaining academic deliverables (College Report, IEEE-style paper, viva prep, demo script, poster — per Master Prompt's Documentation section and Folder Structure §9's `docs/academic/`).
- Final submission/demo rehearsal against the full PRD §6 User Journeys (Primary, Secondary/Guest, Edge Case).

Definition of Done — Milestone C / Final Submission: Every requirement in SRS §3 tagged Must or Should is demonstrably working; every NFR in SRS §5 has been checked at least once against its stated target (even if only via a manual/scripted spot-check rather than full load testing, given academic-scale infrastructure per SRS §5.1's own "academic/demo-scale target" framing).

9. Cross-Cutting Work Not Tied to a Single Sprint

These run continuously across all 8 sprints rather than landing in one:

Activity	Cadence	Owner
Weekly Progress Report (Master Prompt deliverable)	Every sprint, end of week 1 and week 2	Whole team, PM role compiles
CI lint/test on every PR	Continuous	Enforced by <code>.github/workflows/</code> from Sprint 0
<code>job_logs</code> /structured logging review	Continuous, spot-checked each sprint	Whoever owns the AI Pipeline track that sprint
Dictionary curation (adding new slang/profanity terms)	Ongoing, especially after Sprint 6 benchmarking	Whoever has admin access; formalized via Admin Panel from Sprint 7 onward
Coding Standards/Git Workflow adherence (Document per Master Prompt)	Continuous from Sprint 0	Enforced via PR review, documented once in <code>docs/standards/</code>

10. Risk Register

Risk	Likelihood	Impact	Mitigation
Faster-Whisper CPU latency exceeds the $\leq 2\times$ -real-time budget (SRS-NFR-PERF-01) on the team's actual free-tier hosting hardware	Medium	High — undermines the core demo	De-risked <i>first</i> , in Sprint 1, before any other code depends on it (Section 6, Sprint 1's standalone CLI proof-of-concept); fallback is dropping to the <code>small</code> model size (AI Pipeline §19)
Hindi/Hinglish detection recall falls short of the 85% target (PRD §4.6 AC-1)	Medium	Medium — a review-heavy UI (Timeline Editor) is the designed safety net, so this degrades UX rather than breaking the product	Benchmark pass explicitly scheduled in Sprint 6, not deferred to the last week, leaving time to expand the dictionary/seed bank (AI Pipeline §9)
Free-tier hosting quota exhaustion (Render/Railway/Supabase) during the final demo	Low	High — demo-day failure	Separate staging/production environments (System Architecture §6.1) so demo traffic never shares a quota with development churn; final demo rehearsed against production, not staging
Team member availability gaps (exams, other coursework) common in academic timelines	High	Medium	Sprint plan is intentionally sequenced so each sprint's Definition of Done is independently demoable — a slipped sprint delays the <i>next</i> milestone but does not invalidate work already shipped

Risk	Likelihood	Impact	Mitigation
Scope creep toward Future Scope items (voice cloning, live streaming, per Master Prompt's long features list)	Medium	Medium	PRD §9's "Won't Have (This Release)" list is treated as binding for all 8 sprints above; anything not explicitly scheduled here is out of scope until Document 15 (Future Scope)

11. Suggested Role Split (3–5 person team)

Role	Primary sprints	Primary ownership
Backend/API engineer	All	<code>backend/app/</code> — routes, services, repositories, Celery wiring
AI/ML engineer	1-2, 5-6 heaviest	<code>ai-pipeline/</code> — all 9 stages, benchmarking
Frontend engineer(s)	All, especially 3 (Timeline Editor)	<code>frontend/src/</code> — all screens per Doc 10
DevOps/QA (can be shared with another role on a 3-person team)	0, 4, 8 heaviest	CI/CD, deployment configs, regression pass, Test Plan (Doc 13)

On a 3-person team, the AI/ML and Backend roles are the most defensible to merge (both are Python, per Tech Stack §6's own "same-language pipeline" rationale) — the Frontend and DevOps/QA roles are the ones to keep distinct if forced to choose, since the Timeline Editor (Section 6, Sprint 3) and the final regression pass (Section 8, Sprint 8) are each large enough to be a full-time focus on their own sprint.

12. Next Document

Document 12: Implementation Notes — concrete coding-level guidance (naming conventions in practice, example service/repository implementations, dependency-injection wiring patterns) supporting the sprints defined above, followed by Document 13: Testing (Test Plan, Test Cases, QA Checklist) formalizing the regression pass scheduled in Sprint 8.

End of Development Roadmap Document